ANÁLISIS: MODELADO FUNCIONAL, DINÁMICO Y DE OBJETOS

Ingeniería del Software · Carlos Larrosa Sánchez · GII - EUPT · 2026/2027

1. Modelado Funcional

1.1. Diagrama de Casos de Uso

En este apartado se modela la vista funcional externa del “Sistema Inteligente de Progresión Deportiva”, delimitando las funcionalidades con valor observable para el actor principal (Deportista) y los actores secundarios (Amigo y el sistema Calendario Externo). Para garantizar una trazabilidad completa con la Especificación de Requisitos (SRS), se han representado los requisitos funcionales mediante casos de uso en infinitivo, destacando por ejemplo las relaciones obligatorias («include») el cálculo automático de la carga objetivo y la consulta del catálogo de ejercicios al configurar una rutina. Además, aunque la práctica totalidad de las operaciones requieren que el deportista haya iniciado sesión, la relación «include» hacia Autenticarse se ha trazado únicamente en los casos troncales (Gestionar perfil, Gestionar rutinas e Iniciar sesión de entrenamiento) para preservar la legibilidad gráfica del diagrama y evitar el cruce excesivo de líneas. Por último, las acciones condicionales que solo se disparan bajo determinadas circunstancias, como el desbloqueo de logros al superar un hito, se han modelado mediante relaciones «extend».

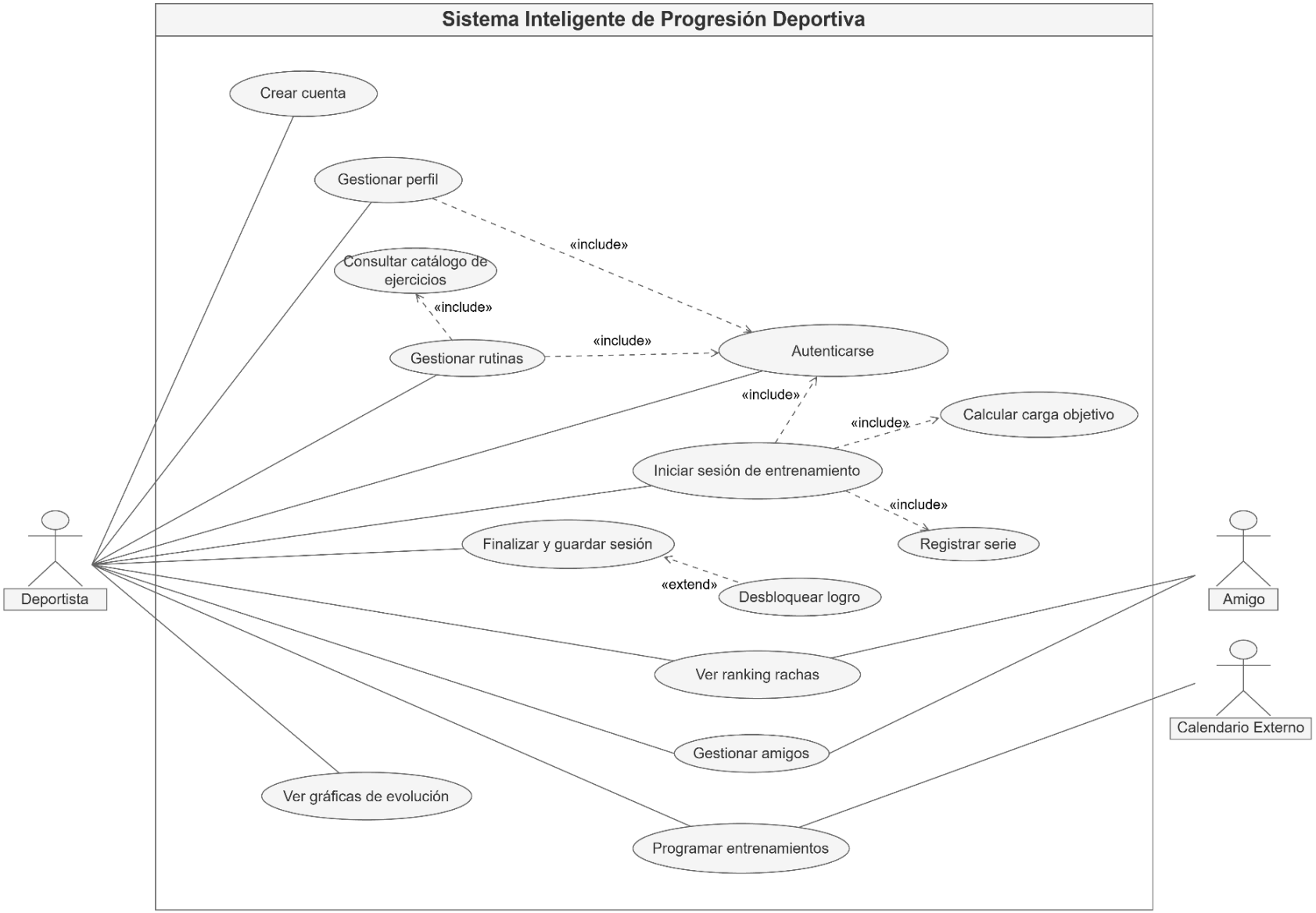


Figura 1. Diagrama general de casos de uso del sistema.

1.2. Diagrama de Actividades

Para el modelado de actividades se ha seleccionado el flujo completo de una sesión de entrenamiento (que integra desde el inicio de la rutina y el cálculo de objetivos hasta el registro de series y el cierre de la sesión, cubriendo del RF-6 al RF-10, RF-12 y RF-15). Se ha elegido este proceso por ser el núcleo de la aplicación, dividiendo el diagrama en particiones (calles) para diferenciar con precisión las acciones que realiza el Deportista frente a los procesos automáticos del propio sistema. El diagrama incorpora bucles anidados para permitir la elección flexible del orden de los ejercicios según la ocupación del gimnasio y la introducción sucesiva de series, decisiones para cubrir tanto el caso de excepción de un ejercicio realizado por primera vez sin historial previo como la cancelación de sesiones en blanco, y barras de concurrencia para ejecutar en paralelo el guardado en el historial, el incremento de la racha y la evaluación de logros.

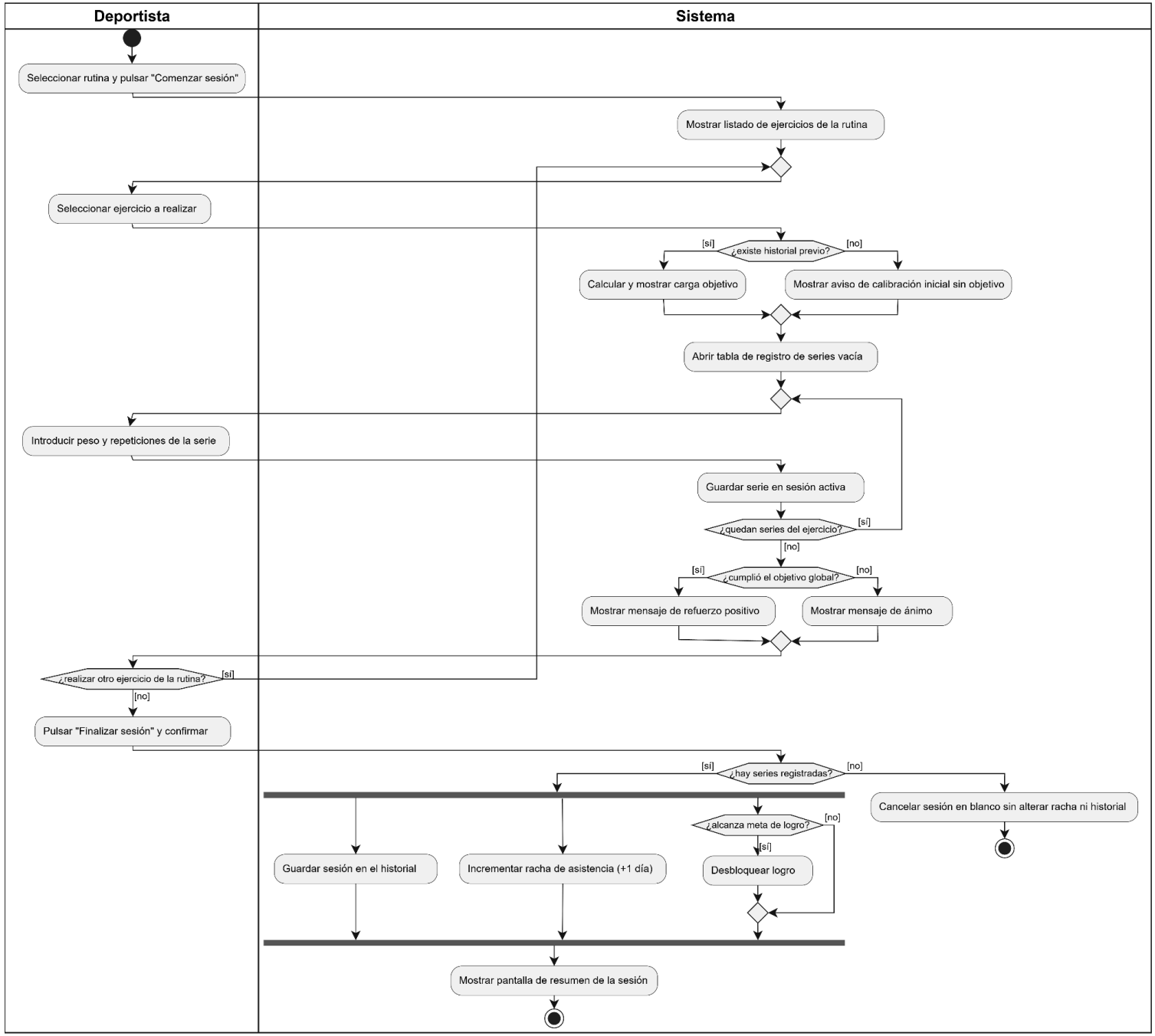


Figura 2. Diagrama de actividades con particiones para el flujo de sesión de entrenamiento y cálculo de cargas.

2. Análisis Dinámico

2.1. Diagramas de Secuencia

Secuencia 1: Iniciar sesión de entrenamiento, calcular carga objetivo y registrar series

Este primer diagrama de secuencia modela la interacción temporal de objetos en el caso de uso central de la aplicación (RF-6, RF-7, RF-8 y RF-12). En el diagrama se diferencian las instancias con estado propio vinculadas al deportista o a la sesión actual (s:Sesion, h:HistorialProgreso, so:SerieObjetivo, sr:SerieReal) respecto de los servicios de cálculo sin estado (CalculadoraMetricas). Mediante un primer fragmento combinado alt se resuelve la bifurcación entre el flujo principal de cálculo de sobrecarga progresiva y la excepción de calibración inicial cuando no existen marcas previas en el historial. Posteriormente, un fragmento loop modela el registro iterativo de cada serie física y un segundo bloque alt determina el mensaje de refuerzo positivo o de ánimo al concluir el ejercicio.

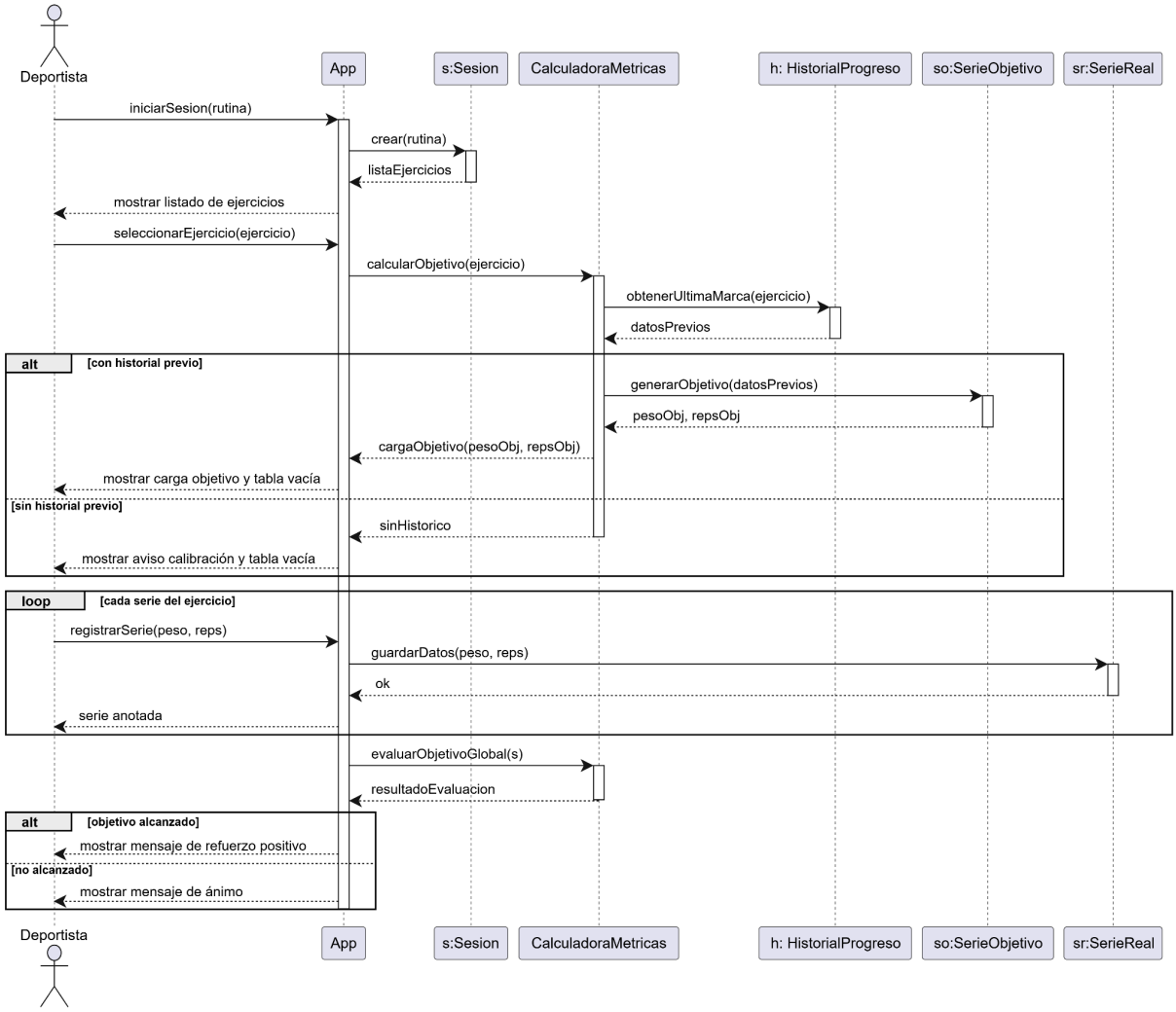


Figura 3. Diagrama de secuencia para el inicio de sesión, cálculo de carga objetivo y registro de series.

Secuencia 2: Finalizar y guardar sesión de entrenamiento

El segundo diagrama detalla el cierre de la sesión de entrenamiento y la activación de los mecanismos de gamificación (RF-9, RF-10 y RF-15). Al solicitar la finalización, el sistema verifica el número de series registradas en s:Sesion y mediante un fragmento alt se separa el flujo principal de la excepción de sesión en blanco. En caso de existir series válidas, se emplea un fragmento combinado par para ejecutar de forma concurrente el almacenamiento en la instancia h:HistorialProgreso del deportista y la actualización de su contador en r:RachaAsistencia, seguido de un fragmento condicional opt que desbloquea el objeto l:Logro únicamente si se alcanza un hito constancia o rendimiento (peso).

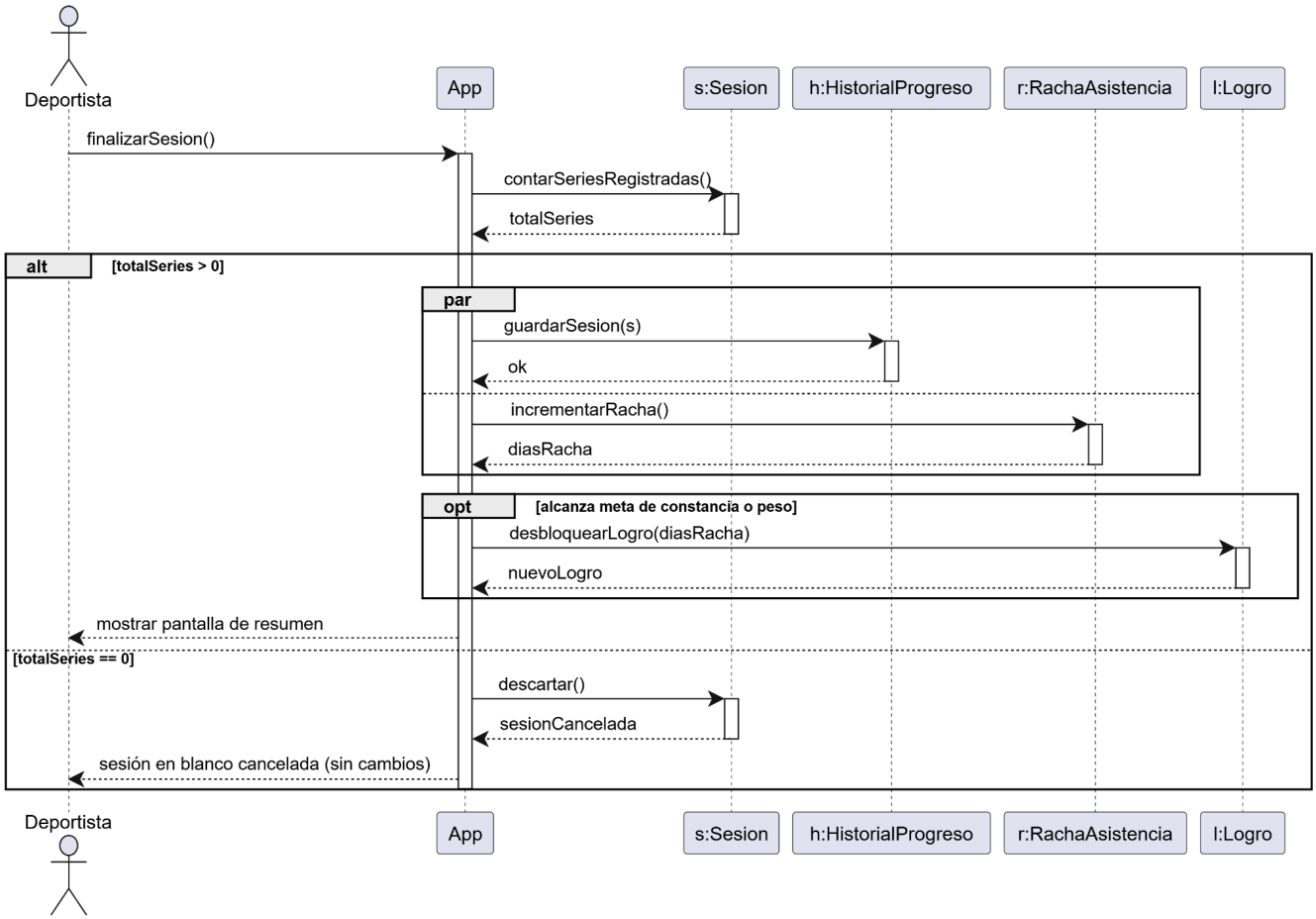


Figura 4. Diagrama de secuencia para la finalización de sesión, actualización de racha y obtención de logros.

Secuencia 3: Programar entrenamientos y sincronizar calendario externo

Con el objetivo de representar la interacción dinámica con sistemas de terceros, se ha incluido este diagrama que muestra cómo el sistema se comunica hacia el exterior modelando la planificación de entrenamientos (RF-16). Cuando el deportista asigna un horario a su rutina, la App delega la conexión en el componente CalendarioExterno, que actúa de puente con el actor externo. Además, se ha incluido un bloque alternativo (alt) para reflejar qué ocurre si falla la conexión a internet en ese momento o se produce un error externo durante la sincronización, cumpliendo así con el requisito de disponibilidad (RNF-4).

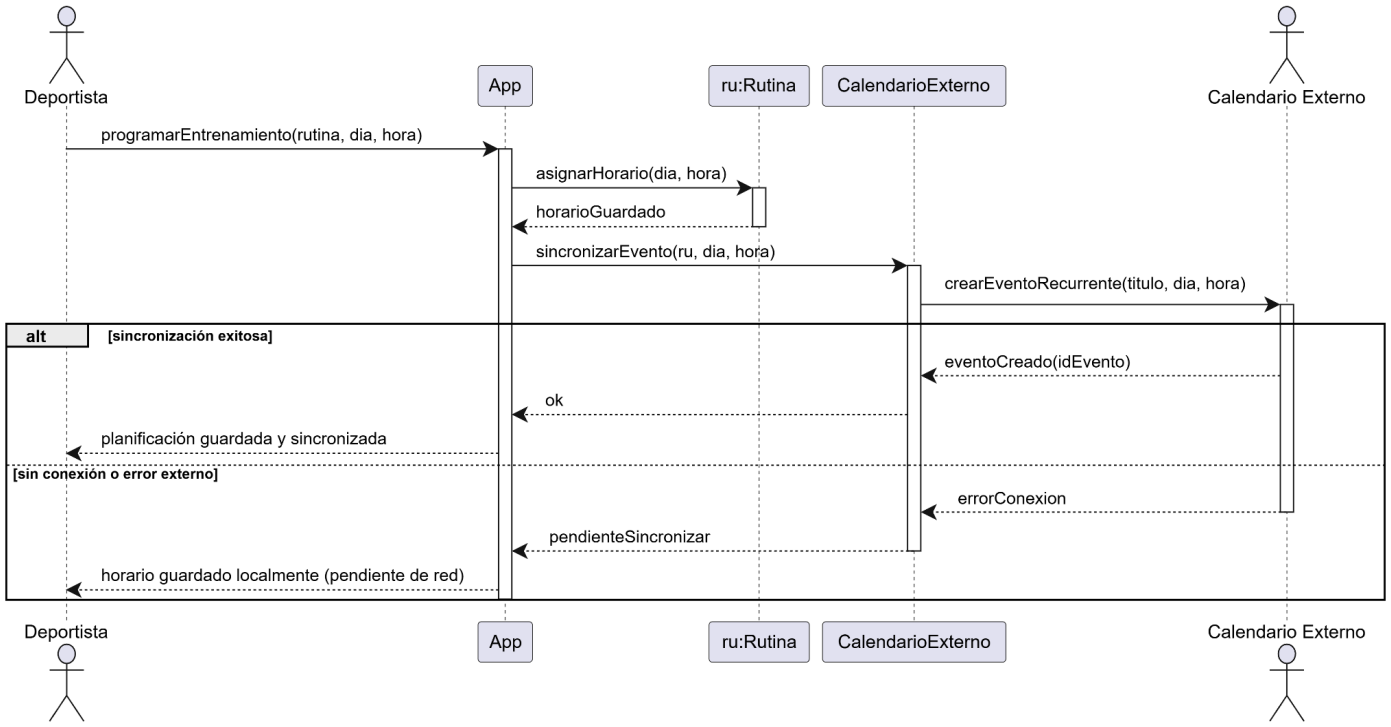


Figura 5. Diagrama de secuencia para la programación de rutinas y sincronización con el calendario externo.

Secuencia 4: Vincular amigo y consultar ranking de rachas

Este diagrama cubre la interacción con el segundo actor secundario (Amigo) dentro del módulo social acotado de la aplicación (RF-13 y RF-14). La línea de vida de la aplicación se divide en tres bloques de activación independientes para reflejar fielmente que el envío de la solicitud (con notificación asíncrona al destinatario), la respuesta diferida del amigo (evaluada con un fragmento alt) y la consulta posterior del ranking (resuelta con un fragmento combinado loop que recopila los días activos de cada instancia r:RachaAsistencia) ocurren en instantes temporales distintos.

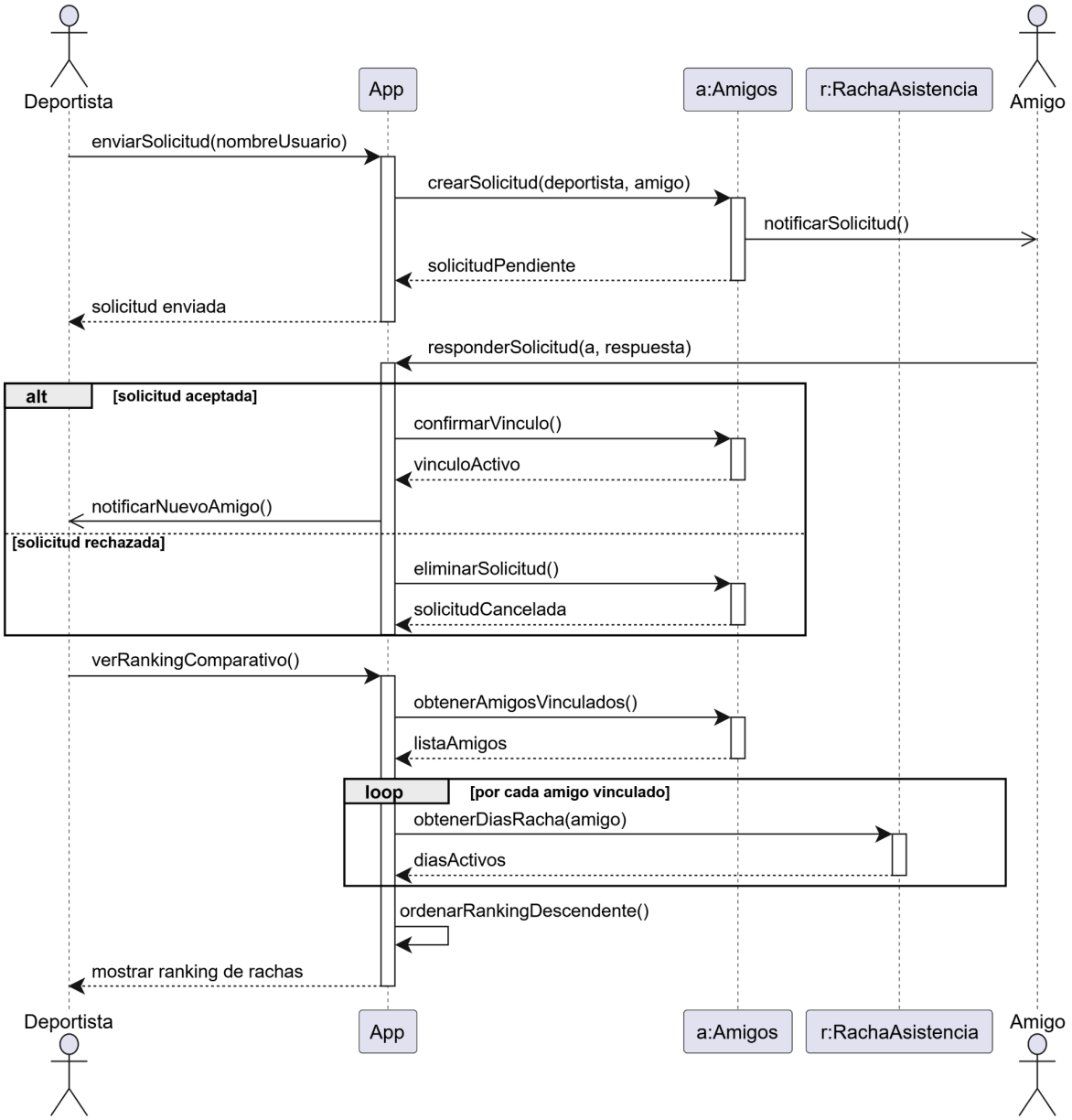


Figura 6. Diagrama de secuencia para la vinculación de amigos y consulta del ranking de rachas.

2.2. Diagramas de Estados

Ciclo de vida del objeto Sesion

Se ha seleccionado la clase Sesion para el modelado de estados por ser la entidad cuyo comportamiento varía de forma más significativa a lo largo de su ciclo de vida. El diagrama recoge el paso desde el estado inicial Preparada hacia EnCurso, e incorpora el estado Interrumpida para dar soporte directo al requisito no funcional de fiabilidad (RNF-5), garantizando que ante un cierre inesperado se conserven las series en almacenamiento local y se recupere la sesión al reabrir la aplicación. Finalmente, los guardas en la transición de salida ([series > 0] frente a [series == 0]) determinan si el objeto culmina su ciclo de vida en el estado Guardada o Cancelada.

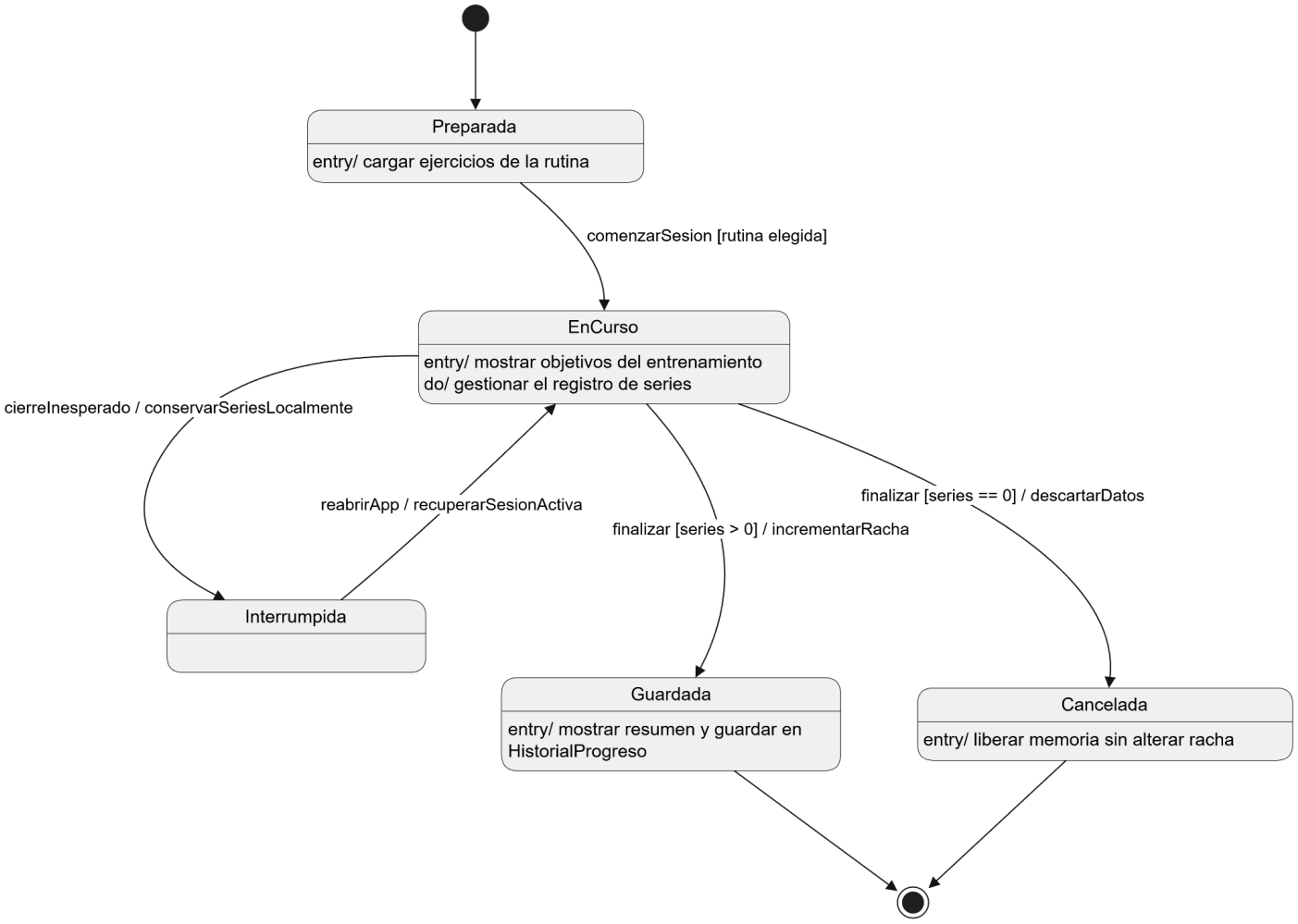


Figura 7. Diagrama de estados correspondiente al ciclo de vida de un objeto Sesion.

Ciclo de vida del objeto RachaAsistencia

Como complemento al modelo dinámico, se representa el ciclo de vida de un objeto RachaAsistencia (RF-10 y RF-14). Este diagrama permite especificar formalmente cómo el contador permanece en el estado Inactiva (con valor cero) hasta registrar la primera sesión válida, se mantiene en estado Activa mientras el deportista cumpla sus entrenamientos dentro del plazo previsto, y regresa a Inactiva reiniciando su racha si vence el plazo sin actividad registrada.

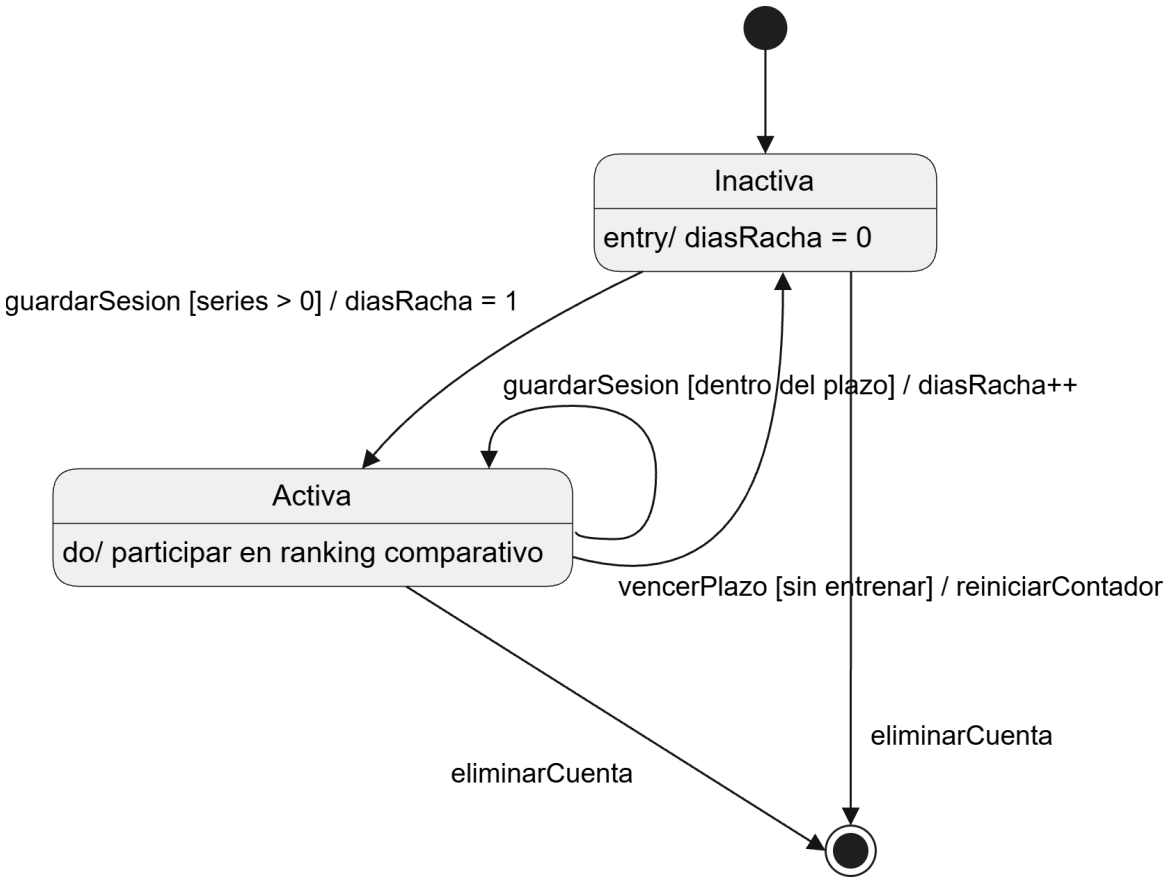


Figura 8. Diagrama de estados correspondiente al ciclo de vida de un objeto RachaAsistencia.

3. Modelado de Objetos

3.1. Diagrama de Clases

Este diagrama define el "esqueleto" de datos del sistema. Se ha utilizado la composición (rombo relleno) para vincular al Usuario con sus rutinas, su racha y su historial, garantizando que si se elimina una cuenta, sus datos desaparezcan en cascada para cumplir con el RF-3 y la normativa del RGPD. En cambio, se aplica la agregación (rombo hueco) entre Rutina y Ejercicio, ya que borrar un plan de entrenamiento no debe eliminar los ejercicios del catálogo general de la aplicación. Por último, para no repetir datos comunes como el peso o el número de repeticiones, se ha creado una clase padre Serie de la que heredan tanto SerieObjetivo como SerieReal, rematando el modelo con sus multiplicidades exactas, la visibilidad de los métodos, la enumeración EstadoSesion y datos calculados al vuelo como el atributo derivado /totalSeries.

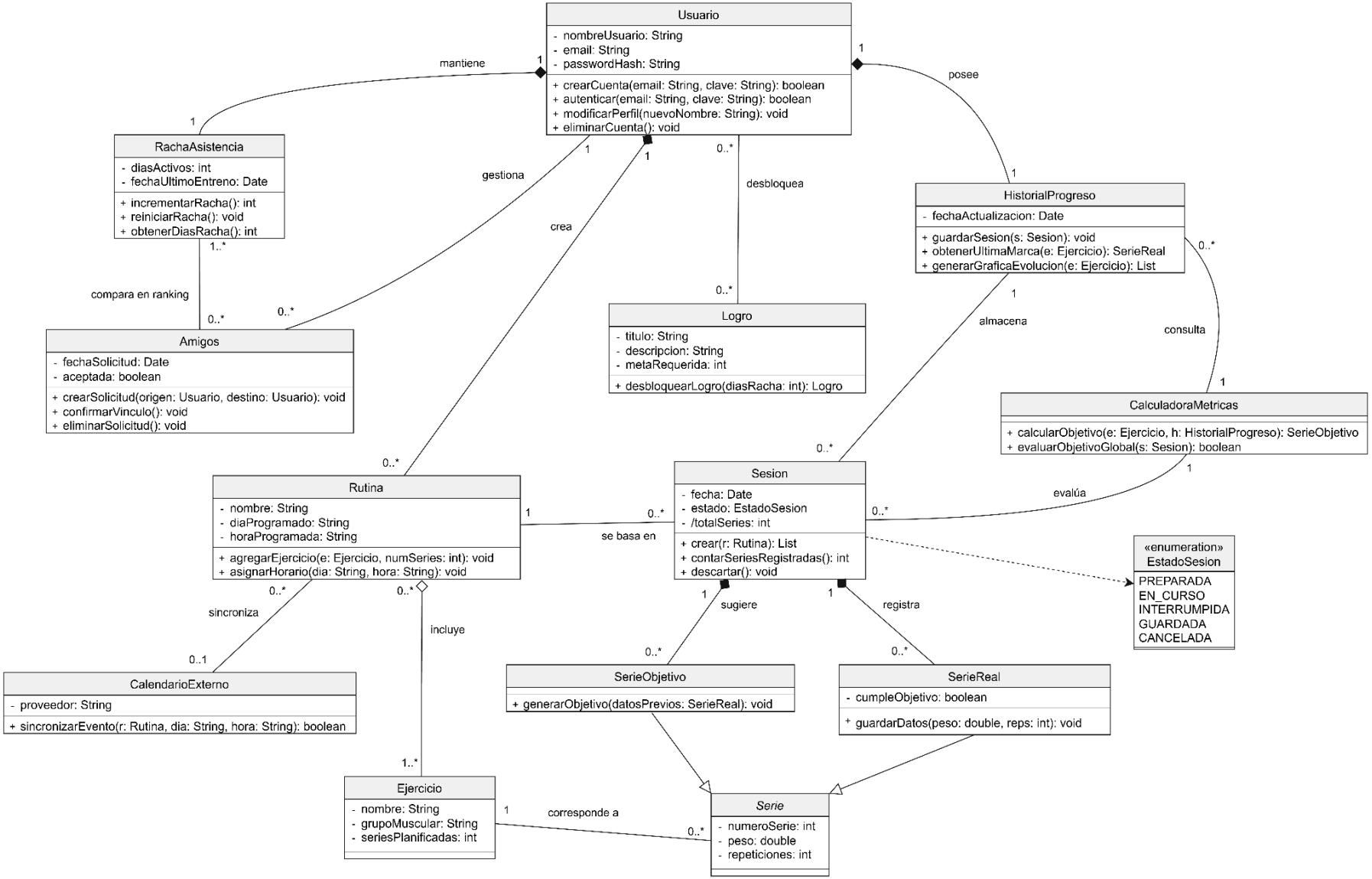


Figura 9. Diagrama de clases del sistema.